TransitPro

Team guide
Who builds what.

Modules, UML diagrams for the CA, screens, deadlines, coding rules and the answers to the lecturer's questions.

PROJECT LEAD
Fanyi Charllson

COURSE
Large System Environment

VERSION
1.1 · 6 November 2026

TEAM
5 members

1 The project in one page

TransitPro is a SaaS platform for intercity bus agencies in Cameroon. Agencies sign up, are verified by us (RCCM, NIU, transport licence), then run their whole business on it: ticket sales, buses, parcels, staff and money. Customers compare verified agencies, book seats and track parcels from their phone.

Five modules share **one central PostgreSQL database**. When something changes in one module, the others see it: a bus in maintenance stops selling seats, every ticket and parcel sale posts to Finance, a driver with an expired licence cannot be assigned to a trip.

PART	WHO USES IT	TECHNOLOGY
Server (`server/`)	Everything talks to it	Express 5, TypeScript, Prisma, PostgreSQL
Web dashboard (`web/`)	Agency office: counter agents, managers, HR, accountant, owner	React 19, Vite, Tailwind v4
Mobile app (`mobile/`)	Customers, plus drivers and couriers in staff mode	Expo SDK 54, Expo Router

Security choices (everyone must respect them)

- Login session in an **httpOnly cookie**: the token never reaches JavaScript, localStorage or AsyncStorage.
- **Role-based access** on every route: Customer, Counter agent, Driver, Courier, Branch manager, Fleet manager, HR officer, Accountant, Agency owner, Platform admin.
- Each agency only sees its own data. Customers can never become staff: staff accounts only come from an HR invite.
- Delivery codes and invite tokens are stored hashed. Public pages show no names or phone numbers.

Links

Repository	github.com/fanyicharllson/transitpro
UI design	claude.ai/artifact/1NiJFaNhTtXLuCb5VuzRaw · pages: Web dashboard, Customer app, Staff app
Coding rules	`AGENTS.md` at the root of the repo. Give it to your AI agent first.
API contract	`docs/API.md` : every endpoint, who can call it, which screen uses it
Database	`server/prisma/schema.prisma` : also the base of the shared class diagram

2 Who does what

Two kinds of work. **UML (CA):** each person draws the 4 diagrams of one module; the lecturer marks each of us on our own. **Building:** three people build the screens, two build the server (the part that saves and checks the data).

FC

Fanyi Charllson Project Lead

UML: Parcel & Courier

BUILDS

Web dashboard (all pages), Parcels server, login and security, merging everyone's work

IN SIMPLE WORDS

Builds what agency staff see on the computer, and the parcel part the lecturer questioned.

Use case: Sender, Receiver, Counter agent, Driver, Courier, Branch manager

Class: Parcel, ParcelEvent, ParcelClaim (+ Escrow, Employee, Branch, Trip)

Sequence: Courier delivers with the receiver's code, escrow released

Activity: Parcel journey with weight check and refusal

AT

Adrien Tello Customer app

UML: Booking & Ticketing (server coded by Joy)

BUILDS

Customer mobile app: login, search, results, agencies, seat, pay, ticket, track parcel. Then web pages.

IN SIMPLE WORDS

Builds what a passenger sees on their phone. Fake data first, real server when Joy's endpoints are ready.

Use case: Passenger, Counter agent, Driver, Payment provider

Class: Route, Trip, Ticket, Review (+ Bus, Payment, User)

Sequence: Passenger books a seat and pays with MoMo

Activity: Search → seat → pay → ticket ("seat taken", "payment failed")

NF

Ngwese Fidel Staff app

UML: Fleet & Maintenance (server coded by Sweet)

BUILDS

Staff mobile app: invite, Courier Today, delivery with the receiver's code, Driver Today, QR scanner. Then web pages.

IN SIMPLE WORDS

Builds what a driver or courier sees on their phone during the day. Fake data first, real server when ready.

Use case: Fleet manager, Driver, Branch manager

Class: Bus, MaintenanceRecord, FuelLog (+ Trip, Employee, Expense)

Sequence: Breakdown → bus in maintenance → trips stop selling → cost to Finance

Activity: Bus maintenance cycle

Joy Server

UML: Human Resources

BUILDS

Server: **HR** (staff, invites, licences, payroll) + **Booking** (trips, seats, tickets)

IN SIMPLE WORDS

Writes the answers the apps ask for: "give me today's trips", "book seat 14", "invite this courier".

Use case: HR officer, Agency owner, Staff member

Class: Employee, StaffInvite (+ User, Role, Branch)

Sequence: HR invites a courier → SMS link → sets password → staff mode

Activity: Assigning a driver to a trip (licence valid?)

Sweet Server

UML: Finance & Reporting

BUILDS

Server: **Finance** (payments, escrow, expenses, daily cash) + **Fleet** (buses, maintenance, fuel, GPS)

IN SIMPLE WORDS

Writes the answers about money and buses: "today's revenue", "this bus is in repair", "cash the courier must hand in".

Use case: Accountant, Branch manager, Courier, Payment provider

Class: Payment, Escrow, Expense, CashRemittance (+ Ticket, Parcel)

Sequence: MoMo payment confirmed by the provider

Activity: Daily cash reconciliation (settled / short)

Why these pairs: Booking needs HR (a driver's licence must be valid before a trip), so Joy has both. Fuel and repair costs go into Finance, so Sweet has both. **For the defense:** Joy walks Adrien through the Booking code and Sweet walks Fidel through the Fleet code, so they can defend their diagrams.

Shared class diagram (whole group, led by Charllson): all tables of the central database and their links. In your sequence diagram, show the call into another module (e.g. `ParcelService → FinanceService`): that link is what the lecturer grades.

3 Joy and Sweet: the server, simply

The apps never touch the database. They ask the server, and the server answers. Each question has an address called an **endpoint**, e.g. `GET /api/booking/trips/search`. Your list is already written in `docs/API.md`, and the empty spots are already in the code. You fill them one by one.

1. Make a branch for the endpoint: `git checkout -b feat/booking-search-trips`
2. **Rules of the input** in `<module>.schemas.ts` (which fields are allowed).
3. **The work** in `<module>.service.ts`: read or save in the database with `prisma`.
4. **The door** in `<module>.routes.ts`: replace the `notImplemented` line.
5. Test with Thunder Client or Postman (log in with a demo account first), run `npm run typecheck`, then open a Pull Request.

Full recipe, first endpoints to build and a ready-to-copy prompt for your AI agent: `docs/SERVER_GUIDE.md`. Look at `src/modules/auth/` for a finished example.

4 Mobile: Adrien and Fidel

One Expo app, two experiences. After login the server says who you are. A customer sees the customer screens; a driver or courier sees the staff screens. Customers never see staff screens. Staff cannot sign up: their agency invites them by SMS.

FOLDER	DESIGN SCREENS	OWNER
`app/(auth)/`	Log in, Create account (Google + email/password)	Adrien
`app/(customer)/(tabs)/`	Home, Trips, Parcels, Profile (bottom bar)	Adrien
`app/(customer)/search/`	Search · where to?, Search results	Adrien
`app/(customer)/agency/`	Verified agencies, Agency profile	Adrien
`app/(customer)/booking/`	Choose a seat, Pay, E-ticket	Adrien
`app/(staff)/(tabs)/`	Courier Today, Driver Today, Scan, Cash, Profile	Fidel
`app/(staff)/deliver/`	Courier delivery: receiver's code, photo, GPS	Fidel
`app/invite/`	Invite · set password	Fidel

Fidel, in simple words: open the design page "Staff app", build each screen in the folder above, show fake data shaped like `docs/API.md`, and switch to the real server when the endpoint is ready. Start with: login → Courier Today → delivery with the code. Setup: `cd mobile`, `npm install`, `npx expo install --fix`, `npx expo start`.

5 Timeline

If the lecturer gives a different CA date, we move the UML weeks.

WEEK	DATES	EVERYONE DOES
1	Nov 5 – 11	Clone the repo, read the plan and the design. Group meeting on the shared class diagram.
2	Nov 12 – 18	Use case + class diagram of your module.
3	Nov 19 – 25	Sequence + activity diagrams. Lead reviews. **CA UML ready.**
4 – 5	Nov 26 – Dec 9	Joy, Sweet: first endpoints. Adrien, Fidel: login and main screens with fake data. Lead: web layout, parcels.
6 – 7	Dec 10 – 23	Screens switch to the real server. First full flow: register parcel → scan → deliver with code.
–	Dec 24 – Jan 3	Holidays: light work only.
8 – 9	Jan 4 – 17	Modules linked (payments, maintenance blocks trips, licence check, live map). Adrien and Fidel help on web.
10	Jan 18 – 24	Test together on real phones, fix bugs, deploy.
11 – 12	Jan 25 – Feb 7	Polish, report, slides. **Defense rehearsal.**

6 The lecturer's questions

If we cannot answer these at the presentation, there is no A. Every member must be able to explain all six answers, not only the Parcel owner.

How do we solve tracking?	Each parcel gets a QR label and a tracking code. Every handover is scanned and saved as a `ParcelEvent` (who, when, where). The bus shares its GPS during the trip. Sender and receiver get an SMS at each step.
How do we make sure the delivery guy delivers?	The parcel always has one named `currentHolder`. Delivery only counts with the **receiver's 4-digit code** sent by SMS, plus a photo and the GPS location. Late parcels raise an alert for the branch manager.
What is sent is what is received?	At registration the agent records declared contents and value, **takes photos before sealing**, closes the parcel with a **numbered seal** and **weighs** it.
Not tampered with?	On arrival the parcel is **weighed again**: more than 0.1 kg difference puts it ON HOLD. The receiver checks the seal number and the photos **before** giving the code, and can refuse (a claim is opened). Optional insurance pays if something is lost.
How do we earn money?	A % commission on every parcel fee, a booking fee per online ticket, insurance %, home delivery fee, cash-on-delivery fee, and a monthly subscription per agency.
Does the payment reach the right person?	Payments go through MTN MoMo / Orange Money. Cash on delivery is held in **escrow** and released **only** when the receiver's code is correct. Cash collected by staff is checked in the daily reconciliation. Everyone gets an SMS receipt.

7 How to start today

1. Accept the GitHub invite, then clone:

```
git clone https://github.com/fanyicharllson/transitpro.git
```

2. Read `AGENTS.md`, `docs/TEAM_PLAN.md` and `docs/API.md`. If you use an AI agent, tell it: "Read AGENTS.md first and only work in my module."
3. Run the server: `cd server`, `npm install`, ask Charllson privately for the `.env` file (never post it in the group), then `npm run db:check` and `npm run dev`.
4. Make a branch for each task: `feat/<module>-<what>`. Never push to `main`. Open a Pull Request; the Lead reviews and merges.
5. Weekly check-in in the WhatsApp group every **Sunday evening**: done, next, blocked. Stuck for more than a day? Say it.